

Language-Aware Probabilistic Tokenization for Multilingual Language Models

Qixuan Shen

University of Southern California

<https://qixuan-rex-shen.github.io>

Abstract

Multilingual language models usually commit to one subword segmentation before contextual encoding. We study whether retaining several plausible segmentations can improve multilingual representations, particularly for code-switched text. We introduce a language-aware probabilistic tokenizer (LAPT) that generates up to three Unigram candidates, aligns their embeddings by overlap in the original character space, and fuses them using tokenizer priors, language evidence, a differentiable language router, and a learned gate. We compare it with deterministic byte-pair encoding (BPE) using matched XLM-R-base backbones trained from scratch on equally sampled English, Spanish, and Hindi CulturaX data. Both arms process 3.2 million sequences and are fine-tuned on four valid GLUECoS task–language combinations over five paired seeds. LAPT improves English–Hindi named-entity recognition accuracy by 0.50 percentage points ($p = .0029$), although its macro- F_1 increase is not significant. Sentiment shows a positive but non-significant trend, and neither POS task improves. The BPE arm also obtains lower aggregate masked-language-model loss. These results provide narrow evidence that tokenization-path fusion can help a semantic sequence-labeling task, but do not support a general advantage across tasks.

1 Introduction

Tokenization is a fundamental component of language models because it defines how raw text is transformed into discrete representations before contextual processing. The tokenizer determines the basic units exposed to the model and affects sequence length, computational efficiency, optimization, and downstream behavior (Kudo and Richardson, 2018b; Ali et al., 2024; Dagan et al., 2024). Recent work shows that tokenizer design can materially affect both the efficiency and effectiveness of model training. It should therefore be considered

part of model design, rather than merely a fixed preprocessing step.

Despite this role, tokenizer design has historically received less attention than scaling, pretraining objectives, and model architecture (Dagan et al., 2024). The relative neglect matters because tokenization fixes the interface through which every later layer observes text; representational distinctions removed at that interface cannot be recovered from the original string by the Transformer.

The problem is particularly important for multilingual language models, where one tokenizer must represent languages with substantial differences in script, morphology, and word formation. Modern multilingual systems generally rely on shared subword tokenization, including Byte Pair Encoding (BPE) and SentencePiece-based methods, to construct one vocabulary across languages (Sennrich et al., 2016; Kudo and Richardson, 2018b; Conneau et al., 2020). This is scalable, but a globally optimized vocabulary need not preserve language-specific units. It can allocate vocabulary capacity unevenly and create large differences in tokenization length and computational cost (Rust et al., 2021; Petrov et al., 2023).

Recent multilingual tokenizer research has consequently emphasized efficiency: reducing token fertility, improving vocabulary allocation, and lowering inference cost (Rust et al., 2021; Petrov et al., 2023; Rana et al., 2026). These metrics are important, but they do not directly test whether a tokenizer improves the quality of representations learned for downstream language understanding. An open question is therefore whether multilingual tokenization can be designed not only to compress text efficiently, but also to preserve linguistic information that improves model behavior.

Most subword pipelines commit to one segmentation before the Transformer sees an input. That decision discards alternatives that may encode useful boundaries or preserve different granulari-

ties. Subword regularization shows that exposing a model to several segmentations across training examples can improve robustness (Kudo, 2018a; Provilkov et al., 2020), while lattice models show that alternative paths can be represented jointly rather than reduced to a single 1-best analysis (Su et al., 2017; Buckman and Neubig, 2018). These observations motivate our central question: can a multilingual model retain and combine several plausible tokenizations in the same forward pass, with their contribution conditioned on language evidence?

We introduce the Language-Aware Probabilistic Tokenizer (LAPT). Instead of selecting one deterministic segmentation, LAPT generates a bounded set of Unigram candidates, assigns probabilities from token and language information, records every token’s span in the original string, and combines candidate embeddings through character-aligned fusion. A frozen character n -gram classifier supplies a language prior, while a differentiable router and learned gate participate in the fused forward computation. The Transformer architecture after the input embedding interface remains unchanged.

We make three contributions. First, we provide an end-to-end implementation of character-aligned, language-conditioned tokenization-path fusion for an XLM-R-style encoder. Second, we conduct a controlled comparison against deterministic BPE with matched data schedules, backbone hyperparameters, optimizer batches, and downstream seeds. Third, we report both positive and negative evidence: a significant English–Hindi NER accuracy improvement, a positive but non-significant sentiment trend, no POS improvement, and weaker aggregate masked-language-model (MLM) loss. The mixed result clarifies where the method currently helps and prevents an unsupported claim of universal superiority.

2 Related Work

2.1 Multilingual Subword Tokenization

Early neural systems commonly used word-level vocabularies, but large inventories and poor treatment of rare and unseen words encouraged a move toward subwords. BPE represents rare words through frequent character sequences (Sennrich et al., 2016), while SentencePiece learns language-independent subword models directly from raw text (Kudo and Richardson, 2018b). These ap-

proaches have become standard in multilingual encoders such as XLM-R, which uses a shared vocabulary across many languages (Conneau et al., 2020). A universal vocabulary, however, creates a trade-off between scalability and linguistic specificity. Languages differ in scripts, morphology, and the amount of meaning expressed at character, morpheme, and word levels; one statistically optimized inventory may not preserve all of those structures equally.

This problem is visible in languages without whitespace-delimited words. Character-based Chinese models can avoid data sparsity and out-of-vocabulary errors introduced by word segmentation (Li et al., 2019), while sub-character representations can expose glyph or pronunciation information below the character level (Si et al., 2023). Chinese segmentation and POS tagging have also been modeled jointly because independent 1-best segmentation can propagate errors into later analysis (Zhang and Clark, 2008). Japanese morphological analysis faces a related need to infer boundaries and grammatical labels from an unsegmented character stream (Neubig et al., 2011). Such work motivates smaller units and representations that preserve relationships among characters, morphemes, and words. These language-specific methods can align token boundaries with local linguistic structure, but their assumptions are difficult to encode in one generic multilingual tokenizer.

Morphologically complex words create a complementary problem. German, for example, productively combines meaningful constituents into long orthographic compounds, while many languages express grammatical and semantic distinctions through derivational or inflectional morphology. Frequency-based subwords need not coincide with those stems and affixes. Morphologically informed segmentation can improve a pretrained model’s interpretation and generalization for complex words (Hofmann et al., 2021). Yet explicit analyzers and rules vary across languages and require additional resources, limiting their use as a universal multilingual solution.

Token-free modeling removes a learned subword vocabulary altogether. T5 established a unified text-to-text Transformer framework while still using subword inputs (Raffel et al., 2020); ByT5 instead processes raw UTF-8 bytes and improves robustness to noise and spelling-sensitive tasks (Xue et al., 2022). Byte sequences are substantially longer than subword sequences, however, so elimi-

nating tokenizer-specific errors introduces a computational trade-off. Recent multilingual tokenizer work instead seeks better vocabulary construction, pre-tokenization, and language balance while retaining efficient subwords (Ali et al., 2024; Dagan et al., 2024; Rana et al., 2026). Our work takes a different direction: it asks whether several valid representations from one fixed vocabulary can be preserved and combined directly for representation quality.

2.2 Probabilistic Tokenization and Linguistic Ambiguity

Traditional subword tokenizers generally return one optimal segmentation, even when several analyses are plausible. Unigram subword regularization models a distribution over segmentations and samples different paths during training (Kudo, 2018a). BPE-Dropout similarly suppresses merge operations so that a word can be represented by different subword compositions (Provilkov et al., 2020). Both results suggest that segmentation diversity can improve robustness. They expose different paths across stochastic training instances, whereas LAPT represents several paths simultaneously within one forward computation.

Ambiguity is especially salient in multilingual text. Identical surface strings can encode language-dependent meanings: for example, English *actual* and Spanish *actual* share an orthographic form, but the Spanish adjective normally means “current” or “present.” In Hindi, the form *kal* can refer to yesterday or tomorrow depending on context. Tokenization alone cannot resolve these semantic ambiguities, but committing early to one segmentation can discard boundary and granularity alternatives before contextual reasoning begins. A probabilistic input representation preserves that uncertainty for the encoder rather than pretending that one locally chosen path is definitive.

2.3 Lattice-Style Tokenization

Tokenization ambiguity is a long-standing issue for languages without explicit word boundaries and for pipelines in which segmentation errors propagate. Joint Chinese segmentation and tagging (Zhang and Clark, 2008) and adaptable Japanese morphological analysis (Neubig et al., 2011) illustrate why boundaries and later linguistic decisions are interdependent. Speech recognition likewise uses lattices to compactly preserve competing hypotheses instead of committing immediately to one

transcript. Lattice representations address the same general problem by retaining multiple paths while preserving their correspondence to positions in the underlying input.

This positional structure matters because candidate tokenizations are not unrelated sequences: they are different analyses of the same characters. Word-lattice encoders can consume multiple Chinese tokenizations instead of a single 1-best segmentation (Su et al., 2017), and neural lattice language models can marginalize over tokenizations at several granularities (Buckman and Neubig, 2018). LAPT adopts the principle of preserving alternatives, but uses a bounded top- K candidate set and explicit original-character overlap. It thereby keeps the Transformer sequence length equal to a designated reference path rather than expanding the encoder into a general lattice architecture.

2.4 Language Awareness and Representation Fusion

Languages differ in vocabulary, morphology, syntax, and semantic convention. Language-conditioned modules such as adapters allow shared multilingual models to retain language-specific behavior rather than treating all inputs as one homogeneous text distribution (Pfeiffer et al., 2020). Soft language information is particularly relevant in code-switching, where several languages may occur within one utterance and language identification itself becomes difficult (Poplack, 1980; Barman et al., 2014). Spanglish and Hindi-English mixing (often called Hinglish) combine lexical and grammatical resources systematically rather than through arbitrary word replacement. Hard assignment of a complete mixed utterance to one language can therefore be inappropriate.

Representation fusion provides a mechanism for retaining complementary information. Languages contribute unequally distributed training data, standardized vocabulary, domain terminology, and structural regularities; collapsing all of these signals into one hard linguistic interpretation may discard useful evidence. Multimodal learning combines representations from distinct modalities (Baltrušaitis et al., 2019), while mixture-of-experts models route examples or tokens among specialized parameter sets (Fedus et al., 2022). Inspired by this broader principle, LAPT fuses alternative tokenization representations according to tokenization priors, language evidence, and learned gates. Unlike a hard language decision, it maintains distri-

butions over English, Spanish, and Hindi. Unlike multimodal or expert routing, however, every candidate describes the same character string and uses the same embedding inventory and Transformer. GLUECoS provides an appropriate evaluation setting because it contains English–Hindi and English–Spanish code-switched tasks (Khanuja et al., 2020).

3 Language-Aware Probabilistic Tokenizer

3.1 Overview

A conventional subword tokenizer represents an input sequence X through a single deterministic path

$$T = \mathcal{T}(X), \quad T = (t_1, t_2, \dots, t_n). \quad (1)$$

This interface is efficient, but discards segmentation alternatives before contextual modeling. In multilingual settings, the preferred boundaries and granularity may depend on the language and script of the input.

LAPT replaces this single decision with a probability distribution over a bounded candidate set

$$\mathcal{C}(X) = \{T_1, T_2, \dots, T_k\}, \quad k \leq K, \quad (2)$$

$$\mathcal{P}(X) = \{(T_1, p_1), (T_2, p_2), \dots, (T_k, p_k)\}, \quad (3)$$

where each T_i is a valid segmentation and $\sum_i p_i = 1$. The candidates are aligned through their original character spans and fused into one embedding sequence. The Transformer therefore receives information from several tokenization paths without changing its layer architecture.

3.2 Language-Aware Routing

Multilingual and code-switched text can contain evidence from more than one language. A lightweight language classifier therefore estimates a soft distribution

$$\begin{aligned} \pi^{\text{prior}}(X) &= P(L | X) \\ &= [P(\text{en} | X), P(\text{es} | X), P(\text{hi} | X)]. \end{aligned} \quad (4)$$

The classifier is a character n -gram SGD model trained only on balanced CulturaX training examples and their source-language labels. It never observes GLUECoS inputs or downstream labels. Before candidate scoring, components below the configured language threshold (0.05) are set to zero and the remaining components are renormalized; if all components would be removed, the original

distribution is retained and renormalized. The resulting distribution is used as a soft routing prior rather than a hard language decision, so mixed inputs need not be assigned to only one language.

The fused model also contains a differentiable neural router. It mean-pools reference-path token embeddings and predicts $r(X) = P_\theta(L | X)$. This router is optimized through the fused MLM forward path and an auxiliary language-classification loss. The frozen classifier supports reproducible candidate selection, whereas the trainable router allows the contribution of language information to adapt with representation learning.

3.3 Probabilistic Candidate Generation

For normalized text X , a shared Unigram tokenizer generates several valid paths. The implementation applies a dynamic-programming beam to each whitespace-delimited word, joins the word-level paths in input order, and retains the best K partial document paths after every join. If a sequence exceeds the model limit, each completed path is truncated to the permitted content prefix before special tokens are added. If $T_i = (t_{i1}, \dots, t_{in_i})$, its segmentation likelihood is summarized by

$$s_{\text{tok}}(T_i | X) = \sum_j \log P(t_{ij}). \quad (5)$$

Each path also receives a normalized three-dimensional evidence vector e_i . This vector uses only script and segmentation properties: Devanagari and Latin character counts, Spanish-marked characters, mean piece length, and piece count. It does not use semantic or task labels. Language compatibility is

$$s_{\text{lang}}(T_i | X) = \langle \pi^{\text{prior}}(X), e_i \rangle, \quad (6)$$

and candidate selection combines the two sources:

$$s_i^{\text{sel}} = \alpha s_{\text{tok}}(T_i | X) + \beta s_{\text{lang}}(T_i | X). \quad (7)$$

The coefficients α and β control tokenization likelihood and language evidence. Both are 1.0 in the completed experiment.

3.4 Candidate Probability Normalization

Scores become probabilities through a temperature-scaled softmax:

$$p_i = \frac{\exp(s_i^{\text{sel}}/\tau)}{\sum_{j=1}^k \exp(s_j^{\text{sel}}/\tau)}. \quad (8)$$

A lower temperature sharpens the distribution; a higher value preserves more probability mass across alternatives. Candidates below threshold ϵ are removed,

$$\mathcal{C}'(X) = \{(T_i, p_i) : p_i \geq \epsilon\}, \quad (9)$$

and retained probabilities are renormalized:

$$q_i = \frac{p_i}{\sum_{j \in \mathcal{C}'(X)} p_j}. \quad (10)$$

The final set satisfies $|\mathcal{C}'(X)| \leq K$. We use $K = 3$, $\tau = 0.7$, and $\epsilon = 0.05$. These are runtime candidate-selection settings; the frozen 50,000-piece Unigram vocabulary is not pruned when K changes.

3.5 Character-Aligned Token Representation

Different candidates contain different boundaries and token counts, so tokens cannot be compared by sequence index. The first retained beam path (candidate 0) defines the reference sequence. For token t_{ij} in candidate T_i , LAPT records a half-open span

$$a_{ij} = [s_{ij}, e_{ij}) \quad (11)$$

in the original input string, with reference positions denoted a_{0r} . Character overlap is

$$\text{ov}(a, b) = \max(0, \min(e_a, e_b) - \max(s_a, s_b)). \quad (12)$$

Candidate embeddings are projected to reference position r by overlap-weighted aggregation:

$$\tilde{h}_{ir} = \frac{\sum_j \text{ov}(a_{0r}, a_{ij}) E(t_{ij})}{\max\left(1, \sum_j \text{ov}(a_{0r}, a_{ij})\right)}. \quad (13)$$

This preserves input order while placing all candidates in a shared reference space. Positions without overlap retain the reference embedding. Special and padding tokens use invalid spans and likewise fall back to the reference representation. During MLM, one set of reference-path mask decisions is transferred to every overlapping candidate token, preventing an unmasked alternate path from revealing a target.

3.6 Language-Conditioned Candidate Fusion

If fusion uses only the normalized tokenizer priors, it reduces to the special case

$$H_r = \sum_i q_i \tilde{h}_{ir}, \quad (14)$$

but the implemented model uses the router and a learned candidate gate to adapt these weights. Let m_{ir} be candidate i 's padded attention-mask entry at index r , applied along the aligned tensor, and let G be a learned affine map. The scalar gate is

$$g_i = G\left(\frac{\sum_r m_{ir} \tilde{h}_{ir}}{\max(1, \sum_r m_{ir})}\right). \quad (15)$$

The aligned input embeddings in this expression already contain the shared dynamic MLM corruption decisions. The final score and weight are

$$z_i = \alpha \log q_i + \beta \langle r(X), e_i \rangle + g_i, \quad (16)$$

$$w_i = \frac{\exp z_i}{\sum_j \exp z_j}, \quad (17)$$

and the Transformer input becomes

$$h_r = \sum_i w_i \tilde{h}_{ir}. \quad (18)$$

Consequently, tokenization priors, multilingual context, and trainable fusion all affect the representations used to compute MLM logits. With reference-path MLM targets Y and source-language label L , the optimized proposed-model objective is

$$\mathcal{L} = \mathcal{L}_{\text{MLM}}(Y) + \lambda_{\text{route}} \mathcal{L}_{\text{CE}}(L), \quad (19)$$

where $\lambda_{\text{route}} = 0.05$ in the completed experiment.

3.7 Integration with the Transformer

The deterministic baseline follows

$$X \rightarrow T \rightarrow E(T) \rightarrow \text{Transformer}, \quad (20)$$

whereas LAPT follows

$$X \rightarrow \{T_i, p_i\}_{i=1}^k \rightarrow H \rightarrow \text{Transformer}. \quad (21)$$

The fused sequence enters the XLM-R-style encoder through `inputs_embeds`; no candidate token IDs are independently passed to the encoder. The base Transformer and MLM head match the BPE arm. The proposed arm adds 150,532 router and gate parameters, approximately 0.12% of the 124,493,648-parameter backbone. This isolates the principal architectural difference to input representation while keeping the contextual encoder comparable.

4 Experimental Setup

4.1 Pretraining Data

We use the English, Spanish, and Hindi partitions of CulturaX (Nguyen et al., 2024). CulturaX is multilingual at the collection level, but its individual partitions are language-separated rather than naturally code-switched. This makes it possible to test whether the proposed mechanism transfers knowledge from distinct-language pretraining to code-switched evaluation. We cap each language at approximately 15.0 GB of raw text: 4,224,996 English documents, 4,244,773 Spanish documents, and 2,038,503 Hindi documents. Text is NFKC-normalized, whitespace is stripped and collapsed, and deterministic SHA-256 assignment creates 90/6.7/3.3 train/dev/test splits.

Tokenizer training draws 1.0 GB of normalized text from each language. The BPE sample produces 222.34M English, 215.39M Spanish, and 252.15M Hindi tokens, corresponding to shares of 32.23%, 31.22%, and 36.55%. Both the BPE and Unigram vocabularies contain 50,000 entries and the same five special symbols.

4.2 Model and Optimization Controls

Both arms use an XLM-R-base configuration (Conneau et al., 2020): 12 layers, hidden size 768, 12 attention heads, feed-forward size 3,072, dropout 0.1, and maximum length 512. We use the architecture only; all weights are initialized and pretrained from scratch. MLM masks 15% of eligible positions. AdamW uses learning rate 10^{-4} , weight decay 0.01, gradient clipping 1.0, and linear decay after 6% warmup. Microbatch size is 8 with four-step accumulation, giving an effective optimizer batch of 32 sequences. We train for at most 100,000 optimizer steps, evaluate and checkpoint every 1,000 steps, and retain the checkpoint with lowest macro validation loss.

The comparison shares the exact document schedule and processes 3.2M sequences in each arm. Different segmentations yield 1.336B BPE tokens and 1.255B proposed-model reference tokens. The fixed seed for tokenizer construction, pretraining, and the language-prior classifier is 42. The language prior uses 25,000 balanced training examples per language, character 2–5-grams, 2^{18} hashing features, and an SGD log-loss classifier. The differentiable router’s auxiliary loss weight is 0.05.

4.3 Downstream Tasks and Statistical Tests

We fine-tune on four GLUECoS (Khanuja et al., 2020) task–language combinations: English–Hindi Romanized NER, English–Hindi Romanized fine-grained POS, English–Spanish POS, and English–Spanish sentiment. Table 4 lists the official split sizes. GLUECoS is deliberately chosen because it tests code-switched inputs even though pretraining uses separate monolingual partitions.

Fine-tuning uses AdamW for three epochs with batch size 8, learning rate 2×10^{-5} , weight decay 0.01, 6% linear warmup, and macro- F_1 checkpoint selection. We run paired seeds $\{1, 2, 3, 4, 5\}$; within each seed the arms share data order and task-head initialization. We report mean and sample standard deviation, along with two-sided paired t -tests over seeds at $\alpha = .05$. The experiment record also contains 10,000-sample paired prediction bootstraps.

We exclude the English–Hindi UD POS configuration before interpreting results. Although its nominal train/dev/test sizes are 1,312/201/225, inspection shows that all 3,029 token labels in the test split are NOUN. The degenerate target distribution makes accuracy and macro- F_1 unsuitable for comparing tokenizers; including it would turn a data-integrity failure into an apparent model result.

4.4 Hardware and Software

The experiment ran on Windows 11 with Python 3.14.6, PyTorch 2.12.1, CUDA 13.0, and one NVIDIA GeForce RTX 4060 Laptop GPU with 8,188 MiB memory. The host uses an AMD Ryzen 7 7840H CPU (16 logical processors) and a KIOXIA KXG80ZNV1T02 NVMe drive. No distributed training was used.

5 Results

5.1 Masked-Language Modeling

Table 1 reports test MLM loss from the best validation checkpoints. The BPE model has the lower macro loss (1.965 vs. 2.441), driven by a much lower Hindi loss. The proposed model is slightly lower on English and Spanish. Because the arms predict tokens from separately learned BPE and Unigram vocabularies, their losses are not strictly the same likelihood factorization; we therefore treat downstream evaluation as the more direct comparison.

Test loss	BPE	LAPT
English	2.8188	2.8096
Spanish	2.4609	2.4529
Hindi	0.6159	2.0612
Macro	1.9652	2.4412

Table 1: Per-language MLM loss; lower is better. Cross-tokenizer losses use different target segmentations and should be interpreted cautiously.

5.2 Downstream Evaluation

Table 2 gives the valid downstream results. On English–Hindi NER, LAPT improves accuracy and micro- F_1 from 92.857 to 93.356, a 0.499-point gain that is consistent across paired seeds ($p = .0029$). The prediction-level bootstrap difference is 0.727 points with a 95% interval of [0.385, 1.098]. Macro- F_1 rises by 1.332 points, but its larger between-seed variance makes the change non-significant ($p = .166$). Thus the strongest evidence is specifically for token-level NER accuracy, not balanced performance across entity classes.

English–Spanish sentiment shows a 1.379-point accuracy increase and a 0.364-point macro- F_1 increase, but neither reaches significance with five seeds ($p = .125$ and $p = .129$). Both POS datasets show small negative differences with non-significant paired tests. We therefore describe sentiment as a trend rather than an established improvement and find no evidence of a POS advantage.

Task	Metric	BPE mean $\pm$ SD	LAPT mean $\pm$ SD	Δ (points)	p
NER EN–HI	Accuracy	92.857 $\pm$ 0.080	93.356 $\pm$ 0.161	+0.499	.0029
	Macro- F_1	16.049 $\pm$ 0.007	17.381 $\pm$ 1.755	+1.332	.1657
POS EN–HI FG	Accuracy	24.082 $\pm$ 0.555	23.931 $\pm$ 0.306	−0.151	.5440
	Macro- F_1	1.885 $\pm$ 0.058	1.841 $\pm$ 0.058	−0.045	.2565
POS EN–ES	Accuracy	11.712 $\pm$ 0.055	11.633 $\pm$ 0.162	−0.079	.4144
	Macro- F_1	1.222 $\pm$ 0.063	1.158 $\pm$ 0.015	−0.064	.1185
Sentiment EN–ES	Accuracy	58.707 $\pm$ 2.716	60.086 $\pm$ 2.411	+1.379	.1254
	Macro- F_1	24.651 $\pm$ 0.725	25.015 $\pm$ 0.627	+0.364	.1287

Table 2: Five-seed downstream test results. Values are percentages; Δ is LAPT minus BPE. p is the uncorrected two-sided paired t -test over seeds. Only NER accuracy is significant at .05.

5.3 Discussion

The downstream results suggest that the value of preserving several tokenization representations depends on the characteristics of the task. The clearest improvement occurs for English–Hindi NER. Entity recognition requires the model to identify meaningful spans and determine their categories from surrounding context; both entity boundaries and surface forms can vary across languages. Romanized Hindi–English names can also exhibit variable spelling and segmentation. By retaining

several boundary hypotheses and combining them through language-aware, character-aligned fusion, LAPT can preserve signals that a single deterministic segmentation may discard. This interpretation is consistent with the significant accuracy and micro- F_1 gain, but it must remain narrow: the macro- F_1 change is not significant, its variance is large, and the low absolute macro score indicates strong label imbalance. The experiment therefore does not establish an advantage for rare entity categories.

Sentiment analysis also depends on contextual and semantic information. Sentiment can be expressed indirectly, through interactions among words, or through language-specific lexical and discourse conventions. A probabilistic fusion mechanism could retain alternative representations of these cues, which is compatible with the observed positive direction for both accuracy and macro- F_1 . Nevertheless, the paired tests are not significant and the confidence intervals overlap substantially. The present result is a trend, not evidence of a reliable sentiment improvement; more seeds or a larger code-switched benchmark would be required to distinguish an effect from fine-tuning variance.

In contrast, neither valid POS task improves. POS tagging often depends more directly on local syntactic patterns and grammatical categories. Additional semantic or boundary information may therefore offer less benefit than it does for span-sensitive or sentence-level semantic decisions. This is a plausible task-based explanation, not a causal conclusion: task type is confounded here with dataset size, language pair, label inventory, and annotation quality. The extremely low POS macro- F_1 scores warrant a further audit of label mappings before strong linguistic conclusions are drawn.

The higher macro MLM loss provides an additional counterweight: fusion is not uniformly superior during pretraining. The Hindi gap may reflect the interaction among Devanagari script, the separately learned Unigram vocabulary, heuristic candidate evidence, and reference-path prediction. CulturaX supplies language-separated pretraining partitions, while English–Hindi NER and POS use Romanized code-switched text. Thus the proposed model is asked to transfer a language-aware mechanism learned mostly from monolingual, script-separated data to a mixed and partly transliterated setting. That transfer is deliberately demanding, but it also limits what can be inferred from the downstream gains.

Limitations

The experiment covers only English, Spanish, and Hindi and uses a single pretraining seed on one 8 GB laptop GPU. Five downstream seeds provide a paired comparison but limited power, particularly for sentiment; the reported p -values are exploratory and are not corrected for multiple comparisons. The NER accuracy gain is significant, but its macro- F_1 gain is not, so the result should not be generalized to all entity classes.

The comparison changes both tokenizer algorithm and input construction: deterministic BPE is compared with Unigram candidate generation plus fusion. It therefore cannot attribute effects solely to language awareness or solely to multiple candidates. The proposed arm also adds a small router and gate, and no ablation separates the frozen language prior, differentiable router, learned gate, or character alignment. Pretraining MLM losses are only partially comparable because the token targets differ.

GLUECoS datasets are small and label-imbalanced. One supplied English–Hindi UD POS test split is degenerate and excluded; the remaining low POS macro- F_1 values also warrant a future audit of tag mappings. CulturaX partitions are monolingual, so the work does not pretrain directly on naturally code-switched text. We did not measure wall-clock throughput, energy, inference latency, or memory overhead in a controlled benchmark. Finally, candidate language evidence uses simple script and segmentation heuristics; this is auditable but may encode brittle assumptions and should be replaced or ablated in broader-language studies.

6 Conclusion

Tokenization is not solely an optimization problem for reducing sequence length; it is also a representation-design choice that can influence downstream model behavior. We presented LAPT, a language-aware probabilistic tokenizer that retains up to three Unigram segmentations, aligns them through original-character spans, and fuses them before an otherwise matched XLM-R-style encoder. The method preserves alternative paths rather than forcing multilingual input into one segmentation before contextual encoding.

In a controlled English–Spanish–Hindi experiment, LAPT yields a significant 0.50-point accuracy improvement for Romanized English–Hindi NER. Sentiment improves numerically but not

significantly, POS does not improve, and aggregate MLM loss favors BPE. The evidence therefore supports a focused conclusion: simultaneous tokenization-path fusion can help at least one code-switched, semantic sequence-labeling setting, but its benefit is task-dependent rather than universal. Future multilingual tokenization research should evaluate representation quality alongside fertility and efficiency, while adding matched Unigram ablations, more pretraining seeds, larger and valid code-switched benchmarks, learned candidate evidence, and controlled measurements of runtime and memory cost.

References

- Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leueling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, Charvi Jain, Alexander Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, and 2 others. 2024. [Tokenizer choice for LLM training: Negligible or crucial?](#) In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3907–3924. Association for Computational Linguistics.
- Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. 2019. [Multimodal machine learning: A survey and taxonomy](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2):423–443.
- Utsab Barman, Amitava Das, Joachim Wagner, and Jennifer Foster. 2014. [Code mixing: A challenge for language identification in the language of social media](#). In *Proceedings of the First Workshop on Computational Approaches to Code Switching*, pages 13–23. Association for Computational Linguistics.
- Jacob Buckman and Graham Neubig. 2018. [Neural lattice language models](#). *Transactions of the Association for Computational Linguistics*, 6:529–541.
- Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. [Unsupervised cross-lingual representation learning at scale](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451. Association for Computational Linguistics.
- Gautier Dagan, Gabriel Synnaeve, and Baptiste Roziere. 2024. [Getting the most out of your tokenizer for pre-training and domain adaptation](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 9784–9805. PMLR.
- William Fedus, Barret Zoph, and Noam Shazeer. 2022. [Switch transformers: Scaling to trillion parameter](#)

- models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.
- Valentin Hofmann, Janet B. Pierrehumbert, and Hinrich Schütze. 2021. [Superbizarre is not superb: Derivational morphology improves BERT’s interpretation of complex words](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3594–3608. Association for Computational Linguistics.
- Simran Khanuja, Sandipan Dandapat, Anirudh Srinivasan, Sunayana Sitaram, and Monojit Choudhury. 2020. [GLUECoS: An evaluation benchmark for code-switched NLP](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3575–3585. Association for Computational Linguistics.
- Taku Kudo. 2018a. [Subword regularization: Improving neural network translation models with multiple subword candidates](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75. Association for Computational Linguistics.
- Taku Kudo and John Richardson. 2018b. [Sentence-Piece: A simple and language independent subword tokenizer and detokenizer for neural text processing](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71. Association for Computational Linguistics.
- Xiaoya Li, Yuxian Meng, Xiaofei Sun, Qinghong Han, Arianna Yuan, and Jiwei Li. 2019. [Is word segmentation necessary for deep learning of Chinese representations?](#) In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3242–3252. Association for Computational Linguistics.
- Graham Neubig, Yosuke Nakata, and Shinsuke Mori. 2011. [Pointwise prediction for robust, adaptable Japanese morphological analysis](#). In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 529–533. Association for Computational Linguistics.
- Thuat Nguyen, Chien Van Nguyen, Viet Dac Lai, Hieu Man, Nghia Trung Ngo, Franck Dernoncourt, Ryan A. Rossi, and Thien Huu Nguyen. 2024. [CulturaX: A cleaned, enormous, and multilingual dataset for large language models in 167 languages](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 4226–4237. ELRA and ICCL.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. [Language model tokenizers introduce unfairness between languages](#). In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.
- Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. 2020. [AdapterHub: A framework for adapting transformers](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 46–54. Association for Computational Linguistics.
- Shana Poplack. 1980. [Sometimes i’ll start a sentence in Spanish y termino en español: Toward a typology of code-switching](#). *Linguistics*, 18(7–8):581–618.
- Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. [BPE-dropout: Simple and effective subword regularization](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892. Association for Computational Linguistics.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *Journal of Machine Learning Research*, 21(140):1–67.
- Souvik Rana, Ashish Kulkarni, Arul Menezes, Chandra Khatri, and Shubham Agarwal. 2026. [MUTANT: A recipe for multilingual tokenizer design](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 46261–46277. Association for Computational Linguistics.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. [How good is your tokenizer? on the monolingual performance of multilingual language models](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3118–3135. Association for Computational Linguistics.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725. Association for Computational Linguistics.
- Chenglei Si, Zhengyan Zhang, Yingfa Chen, Fanchao Qi, Xiaozhi Wang, Zhiyuan Liu, Yasheng Wang, Qun Liu, and Maosong Sun. 2023. [Sub-character tokenization for Chinese pretrained language models](#). *Transactions of the Association for Computational Linguistics*, 11:469–487.
- Jinsong Su, Zhixing Tan, Deyi Xiong, Rongrong Ji, Xiaodong Shi, and Yang Liu. 2017. [Lattice-based recurrent neural network encoders for neural machine translation](#). In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, volume 31, pages 3302–3308. AAAI Press.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.

Yue Zhang and Stephen Clark. 2008. [Joint word segmentation and POS tagging using a single perceptron](#). In *Proceedings of ACL-08: HLT*, pages 888–896. Association for Computational Linguistics.

A Complete Reproducibility Configuration

Code and reproducibility materials are available at <https://github.com/qixuan-rex-shen/multilingual-probabilistic-tokenization>.

Table 3 records the principal configuration used in the completed run. The machine-readable experiment record additionally stores all resolved paths, artifact hashes, per-seed predictions, metrics, and statistical outputs.

B Additional Pretraining Accounting

Both arms complete 100,000 optimizer steps and 400,000 microbatches. Each processes 3,200,000 sequences. BPE processes 1,335,633,540 tokens and selects a checkpoint with macro validation loss 1.9956; LAPT processes 1,255,012,753 reference tokens and selects a checkpoint with macro validation loss 2.4623. The mean reference-candidate weights on the test set are 0.472 (English), 0.444 (Spanish), and 0.464 (Hindi). The associated mean routing losses are 1.75×10^{-4} , 4.78×10^{-7} , and 1.84×10^{-3} .

Component	Setting	Value
Data	Languages; normalization	en/es/hi; NFKC
Data	Raw cap; tokenizer sample	15 GB/language; 1 GB/language
Data	Train/dev/test split	90/6.7/3.3%, SHA-256
Tokenizers	Vocabulary; coverage; min frequency	50,000; 1.0; 2 (BPE)
Candidates	K ; temperature; min probability	3; 0.7; 0.05
Candidates	Language threshold; α ; β	0.05; 1.0; 1.0
Unigram	Shrinking factor; max piece length	0.75; 32
Language prior	Examples/language; n -grams; features	25,000; 2–5; 262,144
Backbone	Layers; hidden; heads; FFN	12; 768; 12; 3,072
Backbone	Max positions; dropout; layer norm ϵ	514; 0.1; 10^{-5}
Pretraining	Objective; mask rate; max length	MLM; 0.15; 512
Pretraining	Batch; accumulation; optimizer steps	8; 4; 100,000
Pretraining	LR; warmup; weight decay; clip	10^{-4} ; 0.06; 0.01; 1.0
Pretraining	Eval/checkpoint interval; retained	1,000; 3
Pretraining	Candidate CPU workers; buffer	2; 2 batches
Fine-tuning	Epochs; batch; LR	3; 8; 2×10^{-5}
Evaluation	Seeds; confidence; bootstrap samples	1–5; 95%; 10,000
Reproducibility	Tokenizer/pretraining seed	42

Table 3: Resolved hyperparameters for experiment experiment_001.

Dataset	Train	Dev	Test
NER EN–HI	2,458	306	308
POS EN–HI FG	2,098	261	264
POS EN–ES	2,167	269	270
Sentiment EN–ES	1,524	184	232

Table 4: GLUECoS examples used in downstream evaluation. The excluded English–Hindi UD POS split contains 1,312/201/225 train/dev/test examples.